P, NP,NP Complete and NP Hard Problems

- All the problems can be categorized into two groups.
- The first group consists of the problems that can be solved in polynomial time by ***deterministic algorithm***(the algorithms which we have been discussing).
  - Linear search - O(n)
  - Binary Search - O(logn)
  - Bubble sort - $O(n^2)$
  - Merge Sort – O(nlogn)
  - Matrx multiplication- $O(n^3)$

  - Polynomial time: $O(n^2)$, $O(n^3)$, O(1), O(n lg n)
  - Not in polynomial time: $O(2^n)$, $O(n^n)$, $O(n!)$

- The second group is consists of problems that can be solved in polynomial time by non-deterministic algorithms.

- For example:
  - 0/1 Knapsack problem $O(2^{n/2})$
  - Travelling Salesperson problem $O(n^2 2^n)$
  - Graph coloring (2^n).
  - Hamiltonian cycle (2^n).
  - Sum of sub sets (2^n).

- ***Definition of P:*** Problems that can be solved in polynomial time by using deterministic algorithms. (“p” stand for polynomial.)

- Definition of NP: Problems that can be solved in polynomial time by using non-deterministic algorithms.

Non Deterministic Algorithm

- Conceptually a nondeterministic algorithm could run on a deterministic computer with an unlimited number of parallel processors.
- Each time more than one step is possible, new processes are instantly forked to try all of them. When one successfully finishes, that result is returned. Thus the computation is very fast.
- Every nondeterministic algorithm can be turned into a deterministic algorithm, possibly with exponential slow down.
- Consider searching an unordered array. A linear search has O(n) expected time. A nondeterministic search has constant expected time.

Non Deterministic Algorithm

- Up to now, the algorithms that we have been using have the property that the result of every operation is uniquely defined.
- Algorithms with this property are called as deterministic algorithms.
- In a theoretical frame work we can remove this restriction on the out come of every operation.
- We can allow algorithms to contain operations whose outcomes are not uniquely defined, but are limited to specified sets of possibilities.
- The machine executing such operations is allowed to choose any one of these outcomes.
- Conceptually, a nondeterministic algorithm could run on a deterministic computer with an unlimited number of parallel processors.

- This lead to the concept of a nondeterministic algorithm.
- To specify such algorithms, we introduce three new functions:

Choice(s) : Chooses one of the elements of set S.

Failure ( ) : Signals an unsuccessful completion.

Success( ) : Signals a successful completion.

- Whenever there is a set of choices that leads to a successful completion, then one such set of choices is always made and the algorithm terminates successfully.
- A nondeterministic algorithm terminates unsuccessfully if and only if there exits no set of choices leading to a success signal.
- The computing time for Choice, Success, and Failure are taken to be O(1).
- A machine capable of executing a nondeterministic algorithm in this way is called a nondeterministic machine.

Example :

- Consider the problem of searching for an element x in a given set of elements A[1:n], n>=1. We are required to determine an index j such that a[j]=x or j=0 if x is not in A. A nondeterministic Algorithm for this is as follows:

```
j:=Choice(1,n);
if A[j]=x then
{
        write( j ); Success();
 }
 else
 {
        write( 0 ); Failure();
 }
```

Nondeterministic sorting

```
Algorithm Nsort( A,n)
// Sort n positive integers
{

     for i:= 1 to n do B[i]:=0;    // initialize B[ ]
      for i:= 1 to n do
      {
                j: = Choice(1,n);
                if( B[j] ≠ 0 ) then Failure();
                B[j]:= A[i];
    }
for i:= 1 to n - 1 do    // Verify order
      if B[i]  > B[ i+1 ]  then Failure ( );

   Write ( B [ 1 : n ) ;
   Success (  );
```

- The nondeterministic algorithms terminates unsuccessfully iff there is no set of choices which leads to the successful signal.

- A machine capable of executing a nondeterministic algorithms are known as nondeterministic machines **(does not exist in practice).**

The *Satisfiability* (SAT) Problem

- *Satisfiability* (SAT):
  - Given a Boolean expression on n variables, can we assign values such that the expression is TRUE?
  - Ex: $((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$
  - Seems simple enough, but no known deterministic polynomial time algorithm exists.

Example: CNF satisfiability

- This problem is in *NP*. Nondeterministic algorithm:
- Example:

  $(A \vee \neg B \vee \neg C) \wedge (\neg A \vee B) \wedge (\neg B \vee D \vee F) \wedge (F \vee \neg D)$
- Truth assignments:

	A	B	C	D	E	F
1.	0	1	1	0	1	0
2.	1	0	0	0	0	1
3.	1	1	0	0	0	1
4.	... (how many more?)					

There are 2^n (Exponential) values.

Nondeterministic satisfiability

Algorithm Eval(E, n)

// Determine whether the propositional formula E is satisfiable. The variables are $x_1, x_2, \ldots x_n$.

```
{    for i:= 1 to n do   // choose a truth value assignment.
     x_i:= Choice( false, true );
     if  E(x_1, x_2,…x_n. ) then  Success( );
     else Failure( );
}
```

Time complexity of the above algorithm is O(n).

Note : above algorithm is a nondeterministic algorithm.

- **Reducibility**
  - A problem Q1 can be reduced to Q2 if any instance of Q1 can be easily reformed as an instance of Q2. If the solution of the problem Q2 provides a solution to the problem Q1, then these are said to be reducible problems.
  - Problem L1 can be reduced to L2 if and only if there is a way to solve L1 by a deterministic polynomial time algorithm using deteministic algorithm that solves L2 in polynomial time.
  - It is represented as L1 α L2
  - This definition implies that if we have a polynomial time algorithm for L2, then we can solve L1 in polynomial time.
  - Example: lcm(m, n) = m * n / gcd(m, n),

Note : All the NP problems can be reducible to satisfiability (SAT)

NP-Hard and NP-Complete Problems

- Let P denote the set of all decision problems solvable by deterministic algorithm in polynomial time. NP denotes set of problems solvable by nondeterministic algorithms in polynomial time.

- Since, deterministic algorithms are a special case of nondeterministic algorithms or NP is a super set of P, $P \subseteq NP$.

The nondeterministic polynomial time problems can be classified into two classes.

1. NP Hard and 2. NP Complete

NP-Hard:

A problem L is NP-Hard iff satisfiability reduces to L (satisfiability α L).

Ex: Halting Problem, Flow shop scheduling problem.

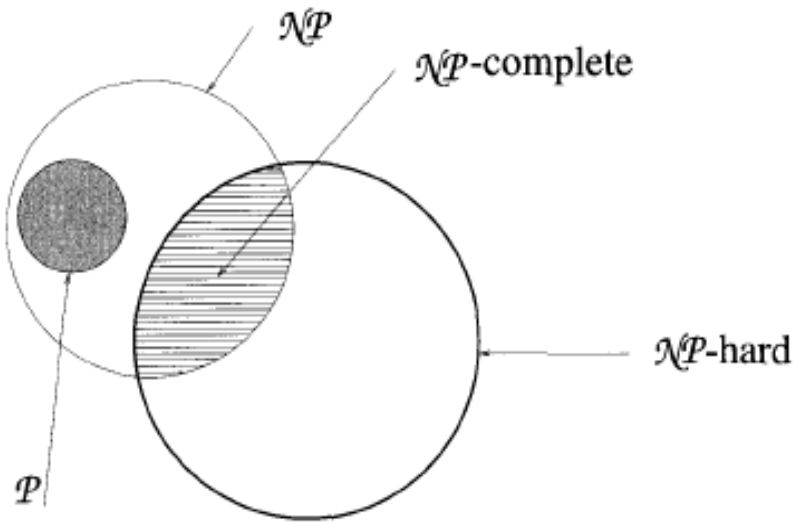
NP-Complete:

A problem L is NP-Complete iff L is NP-Hard and L belongs to NP (nondeterministic polynomial). Examples :

- Boolean satisfiability **problem** (SAT)
- Knapsack **problem**.
- Hamiltonian path **problem**.
- Travelling salesman **problem** (decision version)
- Subgraph isomorphism **problem**.
- Subset sum **problem**.

Note : All the NP- Complete problems are NP-Hard, but some of the NP-Hard problems are not known to be NP-Complete.

Commonly believed relationship among p, NP, NP-Complete, and NP-Hard problems



Cook's Theorem

Cook states that

P = NP iff the satisfiability problem is a *P* problem.

- SAT is NP-complete.
- It is the first NP-complete problem.
- Every NP problem reduces to SAT.
- The most famous unsolved problem in computer science is whether P=NP or P ≠ NP.

Cook states that

P = NP iff the satisfiability problem is a P problem.

- SAT is NP-complete.
- It is the first NP-complete problem.
- Every NP problem reduces to SAT.